Composição paralela como produto monoidal na categoria *Learn*

Kauã Santana da Silva | Alexandre Garcia de Oliveira (orientador)
Fatec Baixada Santista "Rubens Lara" — Tecnologia em Ciência de Dados
kaua.silva@aluno.cps.sp.gov.br

COBRIC 2026

1. Introdução

A Teoria das Categorias oferece uma linguagem para descrever modelos supervisionados por suas interfaces e regras de composição [1]. Em *Learn*, os objetos são espaços de dados e os morfismos são aprendizes (*learners*), que reúnem parâmetros, predição, atualização e requisição.

Essa estrutura não descreve apenas a função preditiva: explicita também como os parâmetros mudam e como uma requisição encaminha um alvo corrigido à entrada. Assim, é possível compor tanto a arquitetura quanto o processo de aprendizado.

Na composição sequencial, as predições se encadeiam; para compor o aprendizado, a requisição também deve propagar o alvo até a entrada. Na composição paralela, dois aprendizes operam em ramos distintos, cada qual com sua entrada e seu alvo.

O objetivo é caracterizar a composição paralela em *Learn* como produto monoidal, demonstrar sua compatibilidade com a composição sequencial pela lei de troca e ilustrar a construção com um bloco residual e com o compartilhamento de pesos em redes convolucionais.

2. Metodologia

A análise é formal e parte da definição de um morfismo $f : A \to B$ como a quádrupla (P_f, I_f, U_f, r_f), em que P_f é o espaço de parâmetros e:

$$I_f : P_f \times A \to B, \qquad U_f : P_f \times A \times B \to P_f, \qquad r_f : P_f \times A \times B \to A.$$

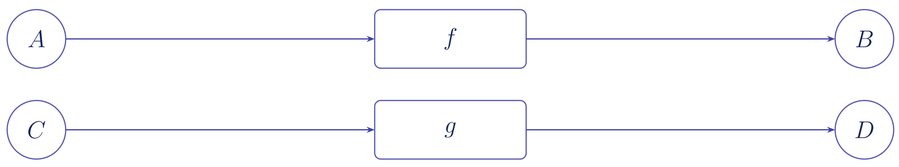
Derivam-se as funções do produto a partir das operações de *Learn*, sob as identificações canônicas dos produtos. Em seguida, aplicam-se cópia e soma ao bloco residual e examina-se a expressão correspondente em Haskell, incluindo as coerções usadas para acomodar os índices de tipos. O escopo é teórico e estrutural: não são avaliados convergência nem desempenho preditivo.

3. Resultados

Produto paralelo

Em *Learn*, o produto monoidal é o produto cartesiano e a unidade é o conjunto unitário. Para $f : A \to B$ e $g : C \to D$, com parâmetros P_f e P_g, obtém-se $f \parallel g : A \times C \to B \times D$ e:

$$P_{f\parallel g} = P_f \times P_g,$$
$$I_{f\parallel g}((p,q),(a,c)) = (I_f(p,a), I_g(q,c)),$$
$$U_{f\parallel g}((p,q),(a,c),(b,d)) = (U_f(p,a,b), U_g(q,c,d)),$$
$$r_{f\parallel g}((p,q),(a,c),(b,d)) = (r_f(p,a,b), r_g(q,c,d)).$$



Cada ramo preserva seus parâmetros e recebe seu próprio alvo; o produto, por si só, não cria comunicação entre eles. Sob as identificações canônicas dos produtos, a lei de troca expressa a compatibilidade com a composição sequencial:

$$(g \circ f) \parallel (g' \circ f') = (g \parallel g') \circ (f \parallel f').$$

Aplicação: bloco residual

A soma μ e a cópia δ têm predições e requisições:

$$I_\mu(a,b) = a + b, \qquad r_\mu((a,b),c) = (c-b, c-a),$$
$$I_\delta(a) = (a,a), \qquad r_\delta(a,(b,c)) = b + c - a.$$

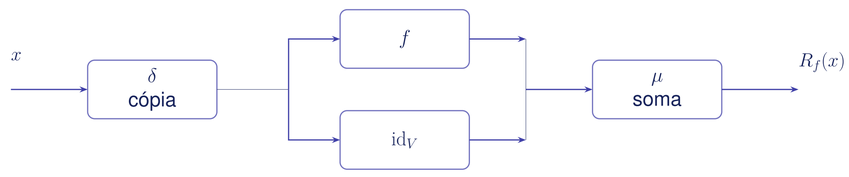
O bloco residual combina um aprendiz com o ramo identidade por meio dessas operações [1,2]:

$$R_f = \mu \circ (f \parallel \mathrm{id}_V) \circ \delta.$$

Para entrada x e alvo y, a predição, a atualização e a requisição resultam em:

$$I_{R_f}(p,x) = I_f(p,x) + x,$$
$$U_{R_f}(p,x,y) = U_f(p,x,y-x),$$
$$r_{R_f}(p,x,y) = r_f(p,x,y-x) + y - I_f(p,x) - x.$$

Assim, o ramo aprendido recebe o alvo residual $y - x$; a requisição da soma e da cópia encaminha a correção até a entrada do bloco.



Implementação e limitação

O operador `(//)` combina learners e concatena suas listas de parâmetros. A assinatura esquemática do produto é:

```
Learner ps A B -> Learner qs C D ->
    Learner (ps ++ qs) (A,C) (B,D).
```

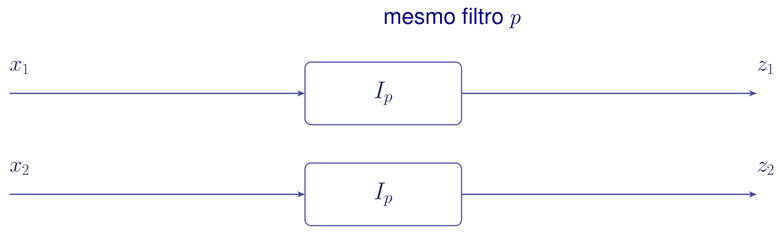
Projeções separam as listas de parâmetros para a predição; cada ramo calcula sua requisição com seus próprios parâmetros, e `unify` recompõe as listas após a atualização. `iniParam` concatena os parâmetros iniciais. O bloco residual é expresso como:

```
residualBlock f = unsafeCoerce(monoid . (f // id) . comonoid).
```

Como a concatenação de listas de tipos não é injetiva, o GHC não prova algumas igualdades de associação nem reduz ps ++ [] a ps quando ps é abstrato. A implementação usa `unsafeCoerce` para contornar essa limitação da representação em Haskell. A coerência entre índices e parâmetros fica sem verificação estática; isso não limita a construção categorial. No bloco residual, a mesma questão aparece ao identificar ps ++ [] com ps.

Compartilhamento de pesos em CNNs

Um mesmo filtro pode processar diferentes regiões de uma imagem. O parâmetro compartilhado é copiado para os learners de cada posição, $p \xrightarrow{\delta} (p,p)$, e as requisições locais são reunidas para informar a atualização comum, $(r_1, r_2) \xrightarrow{\mu} r_1 + r_2$ [1, §V.4]. Diferentemente do produto paralelo simples, os ramos reutilizam os mesmos pesos.



4. Conclusão

A composição paralela é o produto monoidal em *Learn*: parâmetros, predição, atualização e requisição combinam-se componente a componente. A lei de troca a relaciona à composição sequencial. Associatividade, unidade e simetria são satisfeitas sob os isomorfismos canônicos dos produtos. Cópia e soma conectam os ramos no bloco residual e permitem expressar o compartilhamento de pesos. Como continuidade, propõe-se substituir as coerções por evidências de tipos e verificar as leis de identidade e associatividade na implementação.

5. Referências

[1] FONG, B.; SPIVAK, D. I.; TUYÉRAS, R. Backprop as functor: a compositional perspective on supervised learning. In: *Annual ACM/IEEE Symposium on Logic in Computer Science*, 34., 2019. Proceedings [...]. IEEE, 2019. DOI: 10.1109/LICS.2019.8785665.

[2] HE, K.; ZHANG, X.; REN, S.; SUN, J. Deep residual learning for image recognition. In: *IEEE Conference on Computer Vision and Pattern Recognition*, 2016, p. 770–778.